

Agent Native 工作方法论

- Agent 的能力已经够强了，上下文不足才是任务失败的主因。
- 从 Day 0 就用 agent 工作，让上下文随项目自然积累，而非事后补齐。
- Agent = 上下文体 (git 仓库) + 运行时 (任意 coding agent)，框架无关、模型无关。
- 交接项目就是交出上下文体，协作成本趋近于零。
- 对 meta-agent 来说，上下文体最核心的持久层只有两个文件夹：meta-log/ 和 doc/。
- meta-log/ 记录过程，doc/ 沉淀知识；SOP 只是 doc/ 里的一个内容类型。
- doc/roadmap/ 是 doc/ 内用于累积未来功能计划的内容类型，不是第三个核心持久层。

为什么需要 Agent Native

Agent 的工作能力已经够强了。很多时候 agent 完不成任务，不是能力问题，是上下文不够。

大模型的权重只包含通用的专家知识。公司内部的最新项目、客户的态度、开发项目的前因后果、设计上的细微考量，这些从未出现在训练集中的信息，模型没有理由知道。

而把这些上下文传递给 agent 很难。人类只能用文字缓慢输出，带宽就那么大。更麻烦的是，人类往往意识不到自己有多少隐含知识没有传达。就算意识到了，愿意写，持续同步的代价也很大。

所以一个项目跑了一段时间再引入 agent，agent 会面临巨大的上下文缺口，效果很难好。但如果从 Day 0 就用 agent，让 agent 每次工作时自己记录前因后果，上下文就会随项目自然积累。这相当于把文档沉淀的成本均摊到了每次 session 中，每次多记一点，比事后补齐轻松得多。

这是 Agent Native 的第一层意思：从一开始就按 agent 的方式工作，让上下文随项目生长。

但上下文也不是越长越好。单个 session 持续太久，agent 需要同时维持的临时状态、分叉思路、局部结论会越来越多，推理质量容易下降。很多复杂工作不应该硬塞进一个超长 session，而应该拆成多个连续的 session，在边界处做压缩和交接。

用 agent 做协作

Agent 同时也是协作的载体。

传统的工作交接需要写文档、开会、讲前因后果，本质上都是在传递 context，效率很低。用 Agent Native 的方式，交接就是把上下文体交出去。接手的人可以跑一个 coding agent 在这个上下文体上提问，也可以直接翻文件。大部分信息已经在 agent 的记录里了，不需要额外的同步。

Agent 的定义

在这套方法论里，agent 分两部分：

上下文体（静态）是一个 git 仓库，包含项目代码、daily notes、doc、SOP 等全部文件。它可以交接、可以持久化，承载了 agent 跨 session 的连续性。

运行时（动态）是任意 coding agent 程序加上背后的大模型（比如 Claude Code）。运行时在上下文体之上启动，读取信息，执行任务，把成果写回去。

上下文体是通用的，运行时可以换。所以这套方法论不绑定任何 agent 框架或基础模型。

核心组件

对 meta-agent 来说，核心组件只有两个：meta-log/ 和 doc/。

Daily notes / Meta-log

Agent 的工作日志。每次 session 的发现、决策、进展都记在里面，是上下文积累的主要手段。

Append-only，不改历史记录

要更正就显式标注，不要静默修改

每个 session 的记录开头标注操作者（与 agent 协作的人），格式为 operator: <name>。多人协作时可追溯每段记录的主导者。Agent 不知道当前操作者时应主动询问

Doc

项目的知识库，放在 doc/ 目录下。来源不限：agent 的总结、人写的文档、外部参考资料都行。Doc 是对项目某个方面的认知快照。

Roadmap 累积

Roadmap 是 doc/ 下用于保存“现在还不能立即实现，但已经值得提前设计”的未来功能计划的区域。它解决的是单 agent 串行工作时的延迟实现问题：当前 session 继续收束正在做的功能，同时把新的想法转化为可持久化的计划，留给后续时机使用。

宿主项目可以按以下约定组织条目：

```
doc/roadmap/<topic>/plan.md
```

每个条目使用独立目录；plan.md 是推荐入口，但目录内可以自由放置调研、草图、验证结果和其他材料，不强制统一模板、深度、状态或优先级。

Roadmap 的基本循环是：捕获想法 → 用 Plan Mode 或类似流程设计和演练 → 保存计划 → 在当前工作收束后按需阅读 → 由 agent 判断是否开始实现。Roadmap 不是任务队列，不应被自动调度或强制消费。

如果当前 Plan Mode 不允许写文件，应先完成规划，在退出规划模式后保存最终计划。开始实现时可以继续保留和更新原条目；实际执行结果仍写入项目的 meta-log/ 或稳定文档。

SOP

标准操作流程，从 daily notes 的实践中提炼，放在 doc/sop/ 目录下。SOP 不是第三个核心持久层，而是 doc/ 里的一个子类型。SOP 由 agent 执行而不是脚本执行，因为 agent 能处理过程中的意外。每个 SOP 包含：起始条件、结束条件、执行步骤。

SOP 的创建由人决定，agent 可以建议。

工作流程

Agent 由 prompt 启动，prompt 来自用户、自动化系统或仓库中已有的任务上下文，内容是具体的工作任务。Agent 不需要启动时读取所有 daily notes 和 doc，而是根据当前任务按需查阅，自己判断该读什么。

对于接入 meta-agent 的外部项目，建议再加一层运行时入口：把最常执行、最稳定的操作规则投影成宿主项目里的一个短文件，例如 .meta-agent/AGENT-RUNTIME.md。外部 agent 先读这个短文件，再按需打开 meta-agent/doc/methodology.md 看背景和边界情况。这样既保留了 submodule 更新能力，也减少了每次 session 都去翻完整方法论文档的成本。

如果某些步骤已经足够稳定，可以提供很薄的 helper script，专门处理确定性的模板生成等机械动作。helper 不替代 SOP 或 agent 的判断：是否记录、如何取舍、何时开始下一项工作，仍由 agent 基于上下文决定。

信息沉淀的路径可以简单理解为：meta-log/ 记录过程，doc/ 沉淀稳定认知；SOP 和 Roadmap 都是 doc/ 中的内容类型。

Session 的意义

这里的 session，不强调复杂的数据结构，先把它理解成：一个 operator 与一个 coding agent，围绕某个目标，在同一个上下文体上连续协作的一段有边界的过程。

Session 的意义不是形式化管理，而是控制上下文质量。很多任务做不动，不是因为 agent 不会，而是因为当前 session 背着太多临时上下文继续往前拖，开始退化。

所以 session 可以理解为上下文重置单位。一个长任务通常不该要求单个 session 从头扛到尾，而应该拆成多个 session。每个 session 尽量把当前问题推进到更稳定的状态，再交给下一个 session 继续。

这里追求的不是把任务拆得越碎越好，而是让每个 session 都保持在 agent 的有效上下文容量内。只要当前 session 还在稳定推进，就继续；一旦上下文开始失控，就应该结束当前 session，把状态压缩给后续 session。

Roadmap 与下一轮上下文

Roadmap 计划和当前任务入口是两种不同的上下文。Roadmap 保存未来方向的设计，不要求下一轮 session 立即执行；当前任务的进度、结论和下一步仍应写入最新的 meta-log/、目标文件或用户明确提供的任务入口。

如果新想法已经足够清楚，可以先把它保存为 Roadmap 计划，再继续当前 session。只有当某个 Roadmap 条目被 agent 判断为当前最合适的工作时，才把该计划作为本轮任务的补充上下文。

Session 开始流程

每次 agent session 开始时，执行以下流程：

1. 检查最近可执行上下文

先看最新的 meta-log/、当前任务直接引用的目标文件和用户指令。如果其中已经有足够明确的任务入口，就从那里开始。只有在当前任务收束、用户要求继续某个方向、或新想法需要进一步设计时，才按需查看宿主项目的 doc/roadmap/。

2. 如果没有明确入口

等待用户给出指令。

Session 结束流程

每次 agent session 结束时，执行以下标准化流程：

1. 总结写入 daily notes

回顾本次 session 的工作内容，把关键信息写入当天的 daily notes：

先确认当前与 agent 协作的人是谁，并在记录开头写 operator: <name>；如果不知道，必须主动询问，不能用 git config 或其他环境信息替代

做了什么、结论是什么

遇到了什么问题、怎么解决的（或还没解决）

下一步需要做什么

写入的原则和 daily notes 一致：append-only，记录事实和决策，不追求完美。

2. Commit & push

对本文件夹（agent 自己的上下文体所在的 git 仓库）执行 commit 和 push，确保所有变更持久化到远端。这包括 daily notes 的更新、doc 的修改、代码变更等。

3. 判断是否需要后续 agent 接手

Session 结束时，agent 应该主动判断：当前的工作是否已经完成？如果没完成，是否已经收束到适合由后续 agent session 接手继续？

判断依据：

任务是否还有未完成的部分

是否已经形成了更稳定的阶段性结论，而不是停留在混乱中间态

后续 session 是否有明确的下一步可以执行

准备留给下一轮的入口是否比当前上下文更收束

如果判断需要后续 agent 接手当前未完成的工作，应把清晰的进度、下一步和风险写进 meta-log/ 或当前目标文件。Roadmap 只用于尚未排期的未来方向，不代替当前任务的连续性记录。无论记录落在

哪个载体里，都应包含：

任务背景（简要，因为详细上下文在 daily notes 里）

当前进度

具体的下一步指令

需要注意的关键信息或风险

后续 agent 运行在同一个上下文体（同一个文件夹）中，能够访问 daily notes、doc 和所有历史记录。所以这段上下文不需要重复整个历史，只需要给出下一个 session 的清晰入口。

如果任务已经完成，或者后续工作需要人类介入（比如需要做决策、需要外部资源），则在 daily notes 中说明情况；若还有未排期的新方向，再把它保存到 Roadmap。

使用技巧

随时写 daily notes，不要依赖 compact。很多 coding agent 有 compact（压缩上下文）功能，但 compact 要为压缩预留一部分上下文空间。更好的做法是：觉得有东西值得记就立刻写进 daily notes 文件。这样既持久化了信息，又给当前任务腾出了上下文空间，不用等 context 快满了再 compact。

案例：昇腾 910B 集群运维

昇腾 910B 集群上的 DeepSeek-R1 部署与运维项目从 Day 0 就用 agent 驱动。一个月下来积累了：

87 篇 daily notes，覆盖驱动升级、网络调试、性能压测、故障排查的全过程

20 多个 SOP，按阶段组织为环境构建、节点基础设施、网络诊断、部署操作、故障排查五个阶段，有完整的依赖关系图

多份 doc，包括内部性能分析、客户版报告、可行性研究，agent 写的和人写的都有

这里面能看出几件事。87 篇 daily notes 不是专门写的文档，就是每次 session 的工作记录，新的 session 按需翻阅就能拿到历史上下文。SOP 也不是凭空设计的，比如"HCCL Timeout 排查"这个 SOP 就是撞了好几次同样的问题之后总结出来的。整个上下文体通过 git 同步，新的协作者直接对接就行，不用从零了解项目。

常用 prompt 示例

以下是实际使用中常见的几种 prompt，供参考。

开始工作：让 agent 接续之前的任务

看一下最新的 meta-log 和当前目标文件，继续上次停下来的地方。

看一下 meta-log 里最近的记录，继续完成 XXX 的工作。

工作结束：让 agent 收尾

结束 session。

Agent 按照 session 结束流程执行：总结写入 daily notes → commit push → 判断是否需要在当前目标文件或 Roadmap 中留下后续上下文。

中途发现重要信息：立刻记录

这个发现比较重要，马上记到 daily notes 里。

不用等到 session 结束。重要的东西随时写，一方面防丢，一方面释放上下文空间。

记录暂缓实现的功能想法：先做计划再继续当前任务

这个想法暂时不打断当前功能。请先用 Plan Mode 设计和演练，把计划保存到 doc/roadmap/<topic>/plan.md，然后回到当前任务。

后续可以使用：

当前任务已经收束，请查看 doc/roadmap/，判断哪个计划最适合现在开始实现。

让 agent 提炼 SOP

这个流程我们已经做过好几次了，帮我整理成一个 SOP 放到 doc/sop/ 下。

Agent 会回溯 daily notes 里相关的记录，提炼出标准流程。

交接给新协作者

我是新接手这个项目的，帮我了解一下整体情况。

新的人跑一个 agent session，agent 会去读 doc 和 daily notes 来回答。

开放问题：多 agent 协作

有些事情单个 agent 做不了，那就需要多个 agent。多个 agent 之间最大的区别是它们的上下文体不同。

上下文体怎么变得不同的？两种情况：一是两个本来就独立的项目，各自有各自的上下文体，到某天需要协作了；二是一个项目的上下文体越长越大，大到单个 agent 吃不下了，只好拆开。

拆分的判断标准、多 agent 之间怎么协作，目前还没有好的答案。